PALM FRUIT RIPENESS DETECTION

AI-powered image and video detection using YOLOv8

A concise technical documentation prepared from the project report and the supplied implementation screenshots. College-specific material has been intentionally excluded.

3,000	92%	10 FPS
Labeled images	Reported accuracy	Up to

Author Aishwarya H

Repository No GitHub repository link provided

Project focus: automated detection and classification of palm fruit ripeness from uploaded images and videos, with bounding boxes, labels, confidence scores, a Flask web interface, and prediction record storage.

1. Project Overview

Palm Fruit Ripeness Detection is a computer-vision application designed to automate the identification of palm fruit ripeness from visual input. The system uses a trained YOLOv8 model to detect palm fruit bunches and classify them by ripeness level. A Flask-based web application provides the user-facing workflow for uploading images or videos and viewing processed results.

Problem

Manual ripeness inspection is time-consuming and can vary with human judgment, lighting, background conditions, and fruit visibility. The project addresses this by applying deep-learning-based visual detection to the task.

Solution

The application combines annotated training data, image preprocessing and augmentation, YOLOv8 inference, and a web interface. Results are presented visually using bounding boxes, ripeness labels, and confidence scores.

Input	Image or video
AI model	YOLOv8 object detection; CNN-based feature extraction
Classes	Ripe • Semi-ripe • Unripe
Web layer	Flask-based application
Computer vision	OpenCV; NumPy
Dataset tooling	Roboflow for annotation/management
Database	SQL Server prediction records

2. Key Features & User Flow

- Upload an image or video through the web interface.
- Process the input with the trained YOLOv8 model.
- Detect palm fruit regions using bounding boxes.
- Classify detections into ripe, semi-ripe, or unripe categories.
- Display confidence information with the detection result.
- Support image-result and video-result workflows.
- Save prediction information in the SQL Server database.
- Allow processed results to be downloaded where supported by the application.

End-to-End Flow

1 Upload	2 Preprocess	3 YOLOv8	4 Classify	5 Display	6 Store

The report describes the processing sequence as image/video acquisition → preprocessing and augmentation → YOLOv8 training/inference → detection and classification → result display. The supplied UI and database screenshots provide implementation evidence for the upload, result, and storage portions of this flow.

3. AI Model & Dataset

Model: YOLOv8

YOLOv8 is used as the primary object-detection model. In this project it detects palm fruit regions and assigns a ripeness class to each detection. The project report also describes CNN-based feature extraction as part of the model's deep-learning pipeline.

Training Data

3,000	3	92%
Labeled images	Ripeness classes	Reported accuracy

Ripeness Classes

- Ripe — detected as the ripe class.
- Semi-ripe — intermediate ripeness category.
- Unripe — detected as the unripe class.

Preprocessing & Augmentation

- Image resizing and orientation handling.
- Image-quality preprocessing described in the report, including noise removal and contrast adjustment.
- Data augmentation using transformations such as flipping, brightness adjustment, and rotation.
- Bounding-box annotation of palm fruit regions using Roboflow.

Training & Evaluation

The report describes multi-epoch training, monitoring of loss, and evaluation using metrics such as precision, recall, and F1-score. The confirmed project metric used here is 92% reported accuracy; no unsupported precision/recall/F1 numeric values are added.

4. Technical Architecture

Application Architecture

Browser / User	→	Flask Web Application	→	YOLOv8 Inference
Upload image/video		Input handling • result UI		Detection • classification
		↓		
Processed Output	←	Bounding boxes + labels + confidence	←	SQL Server

The project report describes a Flask server handling uploads and passing processed input to the YOLOv8 model. The supplied database screenshot shows a SQL Server database containing prediction records. The screenshot visibly shows fields including an identifier, image path, label, confidence, creation timestamp, and file type.

Implementation Stack

Layer	Technology / Tool	Role
Language	Python	Application and ML development
AI / CV	YOLOv8, CNN, OpenCV, NumPy	Detection, feature learning, image processing
Web	Flask	Upload, inference workflow, result display
Data	SQL Server	Prediction record storage
Annotation	Roboflow	Dataset annotation and management

5. Web Application — UI Screenshots

Home / Upload Interface

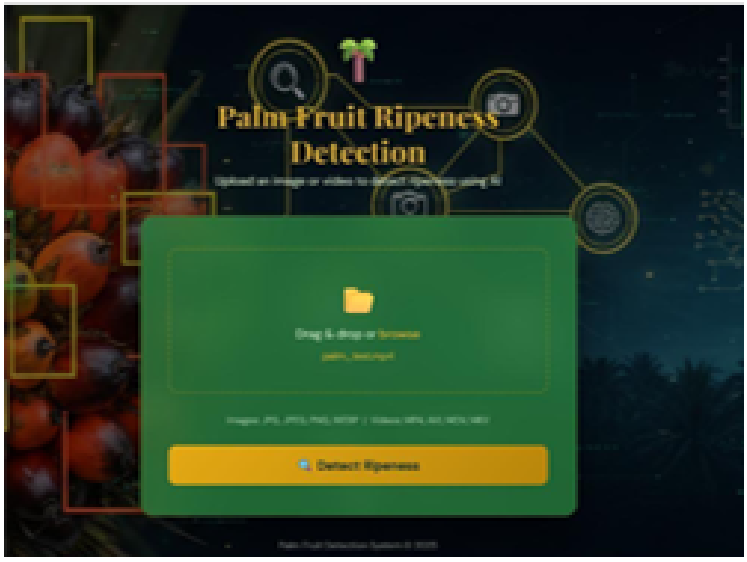


Figure 1 — Home screen showing the Palm Fruit Ripeness Detection interface.

The home screen provides the primary upload workflow and a dedicated action to start ripeness detection.

File Selection & Upload

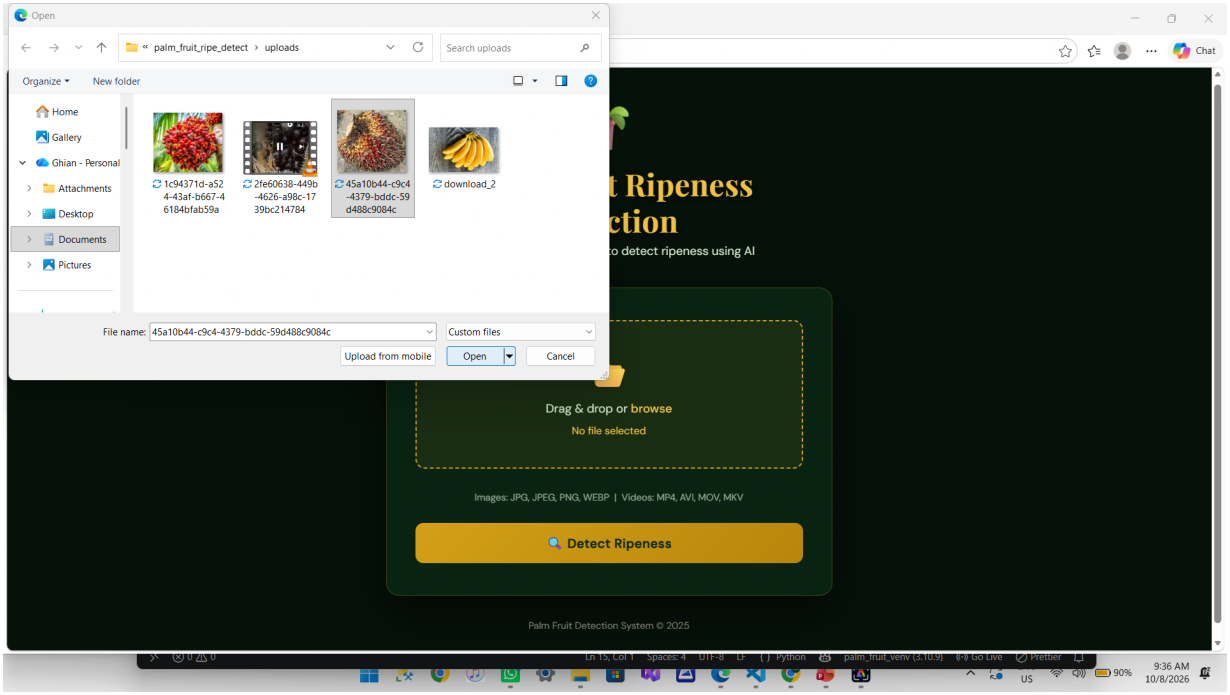


Figure 2 — Image/video selection workflow in the browser.

The supplied implementation screenshot shows image/video selection from the uploads area and the application's Detect Ripeness action. The interface also displays supported image and video formats.

6. Detection Results

Image Detection



Figure 3 — Image detection result with a bounding box, ripeness label, and confidence information.

The image-result screen demonstrates the model output presented through the web application: the original fruit region is annotated with a detection box and classification information.

Video Detection

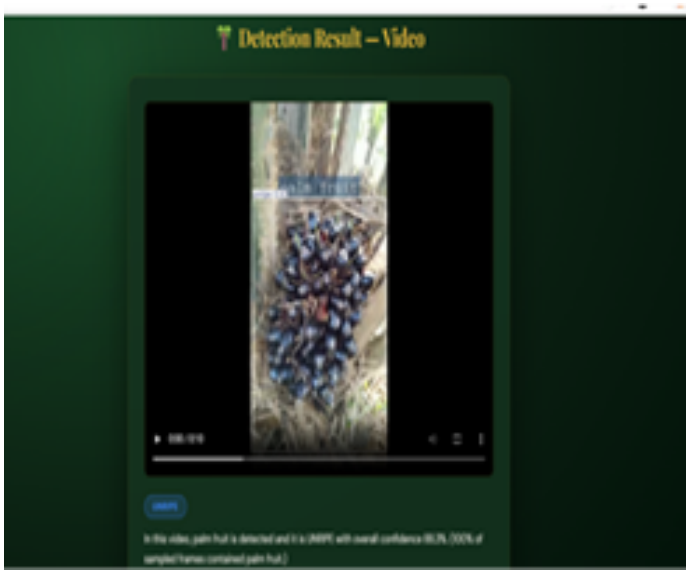


Figure 4 — Video detection result displayed through the web application.

The video-result screen demonstrates the second supported input mode. The project report describes frame-by-frame processing for video inputs.

7. Prediction Storage — SQL Server

The supplied implementation screenshot shows the prediction records in Microsoft SQL Server Management Studio. This confirms that the application includes a database-backed prediction record layer.

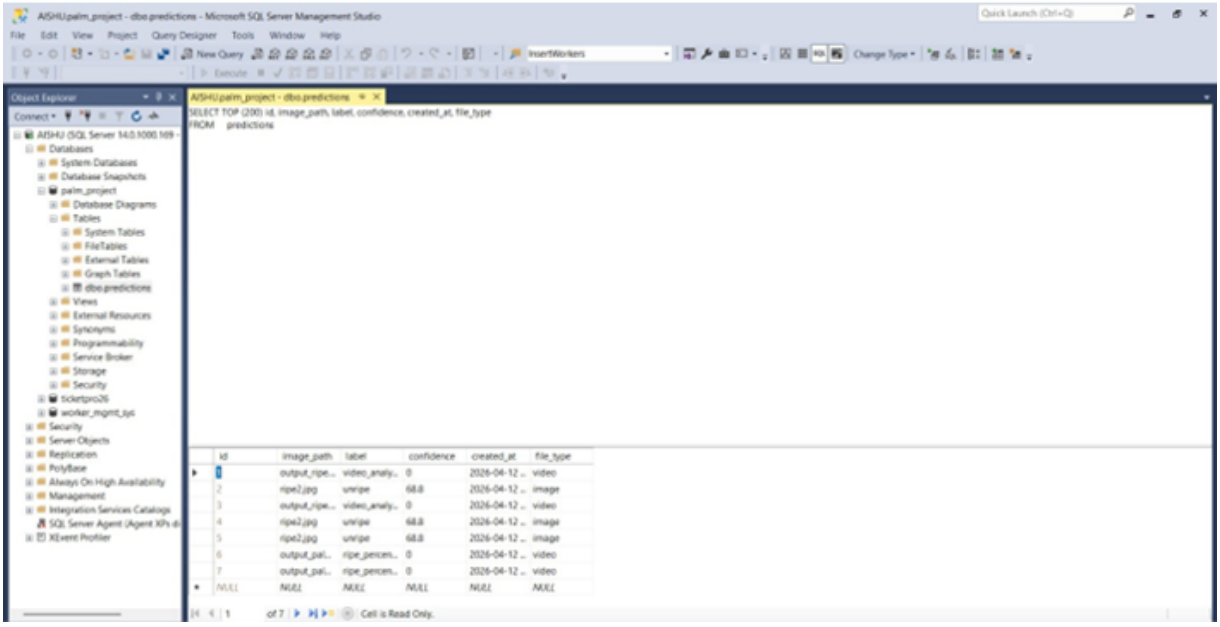


Figure 5 — SQL Server Management Studio screenshot showing prediction records.

Visible Record Fields

Field	Purpose
id	Record identifier
image_path	Stored input/output image path
label	Predicted ripeness label
confidence	Model confidence value
created_at	Prediction record timestamp
file_type	Input/output media type

The field descriptions above are derived from the supplied database screenshot and project report. No additional database schema or relationships are assumed.

8. Implementation & Performance

Core Implementation

- Annotated palm fruit images are prepared for YOLOv8 training.
- Preprocessing and augmentation are used to improve consistency and generalization.
- YOLOv8 performs detection and ripeness classification on new image/video inputs.
- Flask provides the web interface and inference workflow.
- OpenCV is used for image/video processing.
- Prediction details are recorded in SQL Server.
- The interface presents processed results with visual detection annotations.

Confirmed Project Metrics

92%	Up to 10 FPS
Reported model accuracy	Reported processing speed

These figures are included because they were explicitly confirmed for this documentation. The source report itself describes the system qualitatively as accurate and efficient but does not provide the same set of numeric values in the extracted results section.

Observed / Documented Capabilities

- Works with both images and videos.
- Produces bounding boxes, labels, and confidence scores.
- Designed to handle changes in lighting, complex backgrounds, and partially hidden fruits.
- Supports saving/downloading processed results as described in the project report.

9. Limitations & Future Improvements

Current Limitations

- Performance can be affected by lighting, background complexity, image quality, and partially occluded fruit.
- Generalization depends on the diversity and quality of the training data.
- The current implementation is presented as a web application with uploaded media; continuous field-camera monitoring is described as future work.

Future Improvements

- Connect live cameras for continuous plantation monitoring.
- Expand the dataset with more diverse field conditions and retrain the model.
- Integrate cloud storage and analytics for long-term monitoring.
- Explore mobile, IoT, or drone-assisted deployment.

GitHub Project Status

Repository	No GitHub repository link provided yet.
Documentation purpose	Ready to accompany a future GitHub repository as the project's technical overview.
Author	Aishwarya H

Recommended repository sections: README.md • source code • model/weights (or release link) • dataset description • requirements.txt • screenshots • setup instructions • usage examples • results/evaluation.

10. Conclusion

The project demonstrates an end-to-end computer-vision workflow for palm fruit ripeness detection. A YOLOv8 model is trained on a 3,000-image annotated dataset and used to classify palm fruit into ripe, semi-ripe, and unripe categories. The model is integrated into a Flask web application that accepts image and video inputs and presents detection results with visual annotations and confidence information.

The supplied implementation screenshots additionally show a working upload interface, image and video result screens, and SQL Server prediction records. With the confirmed 92% accuracy and up-to-10-FPS processing figure, the project has a clear technical story suitable for a portfolio or future GitHub repository.

Project Palm Fruit Ripeness Detection

Author Aishwarya H

Repository No GitHub repository link provided

Documentation scope: technical project documentation only. College acknowledgements, institution details, literature-survey chapters, roll numbers, and other academic-report material are intentionally omitted.